

Министерство науки и высшего образования Российской Федерации

Дисциплина «Системы искусственного интеллекта»

Модуль 1 «Базы знаний и онтологии»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1

«Создание базы знаний в Prolog и онтологии в Protégé»

Выполнил: Максим Сафин

Группа: [указать группу]

Версия документа: 1.0

Москва, 2026

Аннотация

В работе создана база знаний по предметной области Minecraft на языке Prolog. База описывает мобов, предметы, биомы, измерения и структуры игрового мира. Для получения новых знаний реализованы семь правил логического вывода, включая композицию отношений, конъюнкцию, дизъюнкцию и отрицание. На основе Prolog-модели создана OWL 2-онтология, которую можно открыть и исследовать в Protégé. Онтология содержит иерархию классов, объектные и дататип-свойства, индивидов, ограничения кардинальности и дизъюнктность классов. Корректность моделей проверена автоматическими тестами, структурной валидацией OWL и reasoner HermiT.

Ключевые слова: база знаний, Prolog, OWL, Protégé, HermiT, Minecraft, логический вывод.

Введение

Цель работы - освоить логическое моделирование предметной области и перевод знаний из правил Prolog в онтологическое представление OWL. Базы знаний позволяют отделить знания о предметной области от алгоритма их использования, а правила делают результат вывода прозрачным и проверяемым. Онтология дополняет Prolog-модель формальной иерархией классов, доменами и диапазонами свойств, а также средствами автоматического вывода типов.

В отчёте описаны реализация базы знаний Minecraft, её перенос в OWL 2, проверки и результаты запросов. Система поддержки принятия решений относится к LP2 и не входит в объём данной лабораторной работы.

База знаний в Prolog

Предметная область и словарь

Выбрана предметная область видеоигры Minecraft. Модель включает сущности Mob, Item, Biome, Dimension, Block и Structure. Для мобов заданы свойства поведения `passive/1`, `aggressive/1` и `neutral/1`. Для предметов используются свойства способа получения и типа: `crafted/1`, `dropped/1`, `is_weapon/1`, `is_armor/1`, `is_material/1` и `is_block/1`.

Бинарные отношения связывают сущности: `drops/2`, `spawns/2`, `locates/2`, `generates/2` и `contains/2`. Файл `lab1/knowledge_base.pl` разделён на секции фактов, отношений и правил, снабжён комментариями к каждому разделу. Реализация содержит 21 унарный факт, 18 бинарных фактов и семь правил.

Правила вывода

Правило	Назначение
<code>mob_dimension/2</code>	Находит измерение моба через биом появления.
<code>structure_dimension/2</code>	Находит измерение структуры через биом генерации.
<code>item_dimension_from_structure/2</code>	Находит измерение предмета по структуре, содержащей предмет.
<code>item_dimension_from_mob/2</code>	Находит измерение предмета по мобу, который его оставляет.
<code>available_item_in_dimension/2</code>	Объединяет получение предмета от моба и из структуры.
<code>hostile_mob/1</code>	Определяет потенциально враждебного моба через дизъюнкцию.
<code>non_hostile_mob/1</code>	Определяет невраждебного пассивного моба через отрицание.

Таким образом, правила используют композицию (`spawns -> locates`), конъюнкцию, дизъюнкцию и отрицание. Это покрывает требования к разнообразию логических запросов.

Примеры запросов и ответы

Демонстрация воспроизводится командой:

```
swipl -q -s lab1/defense/run_prolog_examples.pl
```

Запрос	Ответ интерпретатора
<code>passive(sheep).</code>	<code>true.</code>
<code>spawns(Mob, plains).</code>	<code>[sheep,pig,chicken]</code>
<code>drops(sheep, Item), dropped(Item).</code>	<code>[raw_mutton,white_wool]</code>
<code>hostile_mob(Mob).</code>	<code>[zombie,skeleton,zombie_piglin]</code>
<code>non_hostile_mob(Mob).</code>	<code>[sheep,pig,chicken,villager]</code>
<code>mob_dimension(Mob, Dimension).</code>	<code>[sheep-overworld,pig-overworld,chicken-overworld]</code>
<code>item_dimension_from_structure(shulker_shell, Dimension).</code>	<code>[end]</code>
<code>available_item_in_dimension(Item, overworld).</code>	<code>[raw_mutton,white_wool,raw_chicken,carrot,golden_</code>

Онтология в Protégé

Перевод модели в OWL

Файл lab1/ontology/minecraft.owl создан в формате OWL 2 RDF/XML. Корневой класс GameEntity объединяет основные сущности. Классы Mob, Item, Biome, Dimension, Block и Structure образуют второй уровень иерархии. Третий уровень включает, например, PassiveMob, NeutralMob и AggressiveMob, а также CraftedItem, DroppedItem, Weapon, Armor и Material.

Объектные свойства drops, spawnsIn, locatedIn, generatesIn и contains имеют заданные домены и диапазоны. Дататип-свойство stackSize имеет диапазон xsd:nonNegativeInteger. Класс StackableItem ограничен точно одним значением stackSize; каждый Structure имеет хотя бы одно значение generatesIn типа Biome. Основные классы предметной области заданы попарно дизъюнктивными там, где это допустимо.

Компетентностные запросы

После открытия онтологии в Protégé следует выбрать Reasoner > Hermit > Start reasoner, выполнить Classify ontology и открыть вкладку DL Query.

DL Query	Ожидаемые индивиды
PassiveMob	sheep, pig, chicken, villager
AggressiveMob	zombie, skeleton
Mob and (spawnsIn value plains)	sheep, pig, chicken
Structure and (generatesIn value endHighlands)	endCity
Item and (inverse contains some Structure)	carrot, goldenHelmet, shulkerShell
Biome and (locatedIn value nether)	basaltDeltas, soulSandValley

Проверка reasoner

Онтология проверена Hermit 1.4.3.517. Reasoner подтвердил, что owl:Thing is satisfiable. При поиске классов, эквивалентных owl:Nothing, получен только owl:Nothing; следовательно, несогласованных именованных классов нет.

Трассируемость Prolog и OWL

Prolog	OWL-элемент	Комментарий
<code>passive(sheep)</code>	<code>sheep rdf:type PassiveMob</code>	Унарный предикат переведён в принадлежность классу.
<code>is_armor(golden_helmet)</code>	<code>goldenHelmet rdf:type Armor</code>	Тип предмета представлен подклассом <code>Item</code> .
<code>drops(sheep, raw_mutton)</code>	<code>sheep drops rawMutton</code>	Бинарный предикат стал утверждением <code>ObjectProperty</code> .
<code>spawns(sheep, plains)</code>	<code>sheep spawnsIn plains</code>	Связь моба и биома.
<code>locates(plains, overworld)</code>	<code>plains locatedIn overworld</code>	Связь биома и измерения.
<code>stack_size(Item, Count)</code>	<code>stackSize</code>	<code>DatatypeProperty</code> с неотрицательным целым диапазоном.

Тестирование и отладка

Автоматические проверки запускаются из корня репозитория:

```
swipl -q -s lab1/prolog/test/run_tests.pl
bash lab1/ontology/test/validate_ontology.sh
```

Первый набор содержит семь тестов на правила вывода: вывод измерения моба и структуры, получение предметов от мобов и структур, дизъюнкцию источников, враждебность и отрицание. Второй набор проверяет существование OWL-файла, корректность XML и обязательные классы, свойства, индивида и ограничения. В GitHub Actions эти проверки и демонстрационный сценарий Prolog запускаются автоматически для pull request и после merge в main.

Оценка и интерпретация

Prolog удобен для краткой записи выводимых правил: например, определение измерения моба требует лишь композиции двух отношений. OWL и Protégé удобнее для описания таксономии, ограничений на свойства и проверки согласованности reasoner'ом. Ограничение текущей модели состоит в небольшом числе индивидов и в отсутствии временных характеристик игровых объектов. Развитие работы может включать новые измерения, рецепты крафта, параметры мобов и использование БЗ в рекомендательной системе ЛР2.

Заключение

В ЛР1 реализованы база знаний Minecraft в Prolog и согласованная OWL-онтология для Protégé. База содержит факты, отношения и семь правил, а её запросы и выводы покрыты автоматическими тестами. Онтология воспроизводит ключевые сущности и отношения Prolog, добавляет иерархию классов, ограничения и проверку HermiT. Полученная модель является прозрачной, проверяемой и может служить основой для дальнейшей системы поддержки принятия решений.

Приложение: структура репозитория

Файл	Назначение
lab1/knowledge_base.pl	Факты, отношения и правила Prolog.
lab1/prolog/queries/examples.pl	Интерактивные примеры запросов.
lab1/prolog/test/run_tests.pl	Запуск семи тестов Prolog.
lab1/ontology/minecraft.owl	OWL 2-онтология Minecraft.
lab1/ontology/test/validate_ontology.sh	Структурная проверка OWL.
lab1/defense/run_prolog_examples.pl	Воспроизводимый листинг запросов и ответов.